# Docker 卷与 Docker Compose：Cars CRUD Workshop

目标：理解持久的容器存储，然后使用 Docker Compose 以单独的 MySQL 数据库容器运行现有的 FastAPI + 纯 HTML Cars CRUD 应用。

这是 DOCKER_GUIDE.md 的后续。你应当已经安装 Docker，并且构建过
一次 Cars CRUD Docker 镜像。请在项目根目录（cars-crud/）运行命令。

> 两个阶段、两种数据库：第一部分在单个容器中使用原有的 SQLite 应用。第二部分
> 将运行中的应用改为两个容器中的 MySQL。SQLite 数据不会自动
> 迁移到 MySQL。练习时可以从一个全新的 MySQL 数据库开始。

---

# 第一部分 - Docker 卷

## 1. 为什么需要卷

容器有一个可写的文件系统层，但当容器被删除时，该层也会被删除。
重建镜像或重新创建容器不应抹掉重要数据，如车辆记录、用户
上传或数据库文件。

Docker 卷将数据与容器分开存储。Docker 管理其位置和生命周期。
新容器可以挂载同一个卷，并看到上一个容器创建的文件。

```
Without a volume:
  container -> /data/cars.db -> remove container -> database lost

With a named volume:
  container A -> /data/cars.db -> [cars-data volume]
  remove A                              |
  container B -> /data/cars.db --------+   same saved data
```

| 存储类型 | 是什么 | 典型用途 |
|---|---|---|
| 命名卷 | Docker 管理的持久化目录 | SQLite/MySQL 数据、上传文件 |
| 绑定挂载 | 将你电脑上的特定文件夹映射<br>到容器中 | 本地编辑源文件 |
| 匿名<br>卷 | Docker 管理的、未指定名称的<br>卷 | 短期或由镜像定义的<br>存储 |
| tmpfs 挂载 | 保存在内存中的文件（Linux 容器） | 临时敏感或<br>一次性数据 |

卷不是备份。如果数据丢失会造成影响，请另做备份。不要将
同一个活动 SQLite 文件作为可写数据库挂载到多个应用容器中。

## 2. 给我们的 SQLite 容器一个命名卷

DOCKER_GUIDE.md 中的 Dockerfile 已经包含：

```
RUN mkdir -p /data && chown 10001:10001 /data
USER 10001:10001
ENV CARS_DB_PATH=/data/cars.db
```

这使得应用将 SQLite 数据存储在容器内的 /data/cars.db。我们将把
卷挂载到 /data。

如果需要，先构建应用：

```
docker build -t cars-crud:local .
```

对于本地访问 Docker Hub 较慢的机器，请按之前的
指南使用课程的 Azure 代理：

```
docker build --build-arg
BASE_IMAGE=efreiwut.azurecr.io/cache/library/python:3.12-slim -t cars-
crud:local .
```

创建命名卷（首次使用时 Docker 也会自动创建）：

```
docker volume create cars-data
```

如果你之前的 cars-app 容器已存在，请在复用其名称之前停止并删除该容器。
不要删除 cars-data 卷。

```
docker stop cars-app
docker rm cars-app
```

上述两条命令仅当 cars-app 已存在时适用；否则跳过。

运行应用：

```
docker run -d --name cars-app \
  -p 127.0.0.1:8000:8000 \
```

```
  --mount type=volume,source=cars-data,target=/data \
  cars-crud:local
```

打开 http://localhost:8000/ 并创建一辆车。你也可以使用 http://localhost:8000/docs。

关键部分是：

```
source=cars-data    -> the Docker volume holding the data
target=/data        -> where the volume appears inside the container
```

Windows 提示：上面的多行命令使用反斜杠，适用于 macOS/Linux shell。在 PowerShell 上，请将完整的 docker run 命令放在一行，或使用 PowerShell 的反引号续行符。

## 3. 证明车辆在替换容器后仍存在

```
docker stop cars-app
docker rm cars-app

docker run -d --name cars-app \
  -p 127.0.0.1:8000:8000 \
  --mount type=volume,source=cars-data,target=/data \
  cars-crud:local
```

刷新页面。你创建的车辆应当仍在，因为它存储在 cars-data 中，而不是在被删除的容器中。

## 4. 需要记住的命令

| 命令 | 用途 |
| --- | --- |
| `docker volume create NAME` | 创建命名卷 |
| `docker volume ls` | 列出卷 |
| `docker volume inspect NAME` | 检查其配置和挂载位置 |
| `docker inspect cars-app` | 查看容器的 Mounts 部分 |
| `docker exec -it cars-app sh` | 在运行中的容器中打开 shell |
| `docker volume rm NAME` | 在容器删除后删除卷（使用它的容器被删除后） |
| `docker volume prune` | 删除未使用的本地卷；可能具有破坏性 |

| 命令 | 用途 |
| --- | --- |
| `docker run --mount type=volume,source=NAME,target=/data ...` | 挂载命名卷 |
| `docker run -v NAME:/data ...` | 命名卷的较短等价写法 |

实用检查：

```
docker volume ls
docker volume inspect cars-data
docker inspect cars-app
```

注意：docker volume rm cars-data 会删除已保存的 SQLite 数据库。而 docker rm cars-app 本身不会删除命名卷。删除前务必确认哪个卷保存了你的数据。

**Bind mount comparison:** `--mount type=bind,source=/absolute/host/path,target=/data` 使用你在电脑上选择的文件夹。它适用于源代码，但可移植性较差，且可能导致权限问题。Cars 应用以用户 10001 运行，因此本练习使用命名卷更简单。

---

# 第二部分 - Docker Compose 与 MySQL

## 5. Docker Compose 的作用

Docker Compose 在一个 YAML 文件（通常名为 compose.yaml）中描述多个服务。它可以构建镜像、拉取已有镜像、创建容器、在网络中连接它们、挂载卷，并用一条命令启动或停止整个应用。

对于我们更新后的应用：

```
Browser
  |
  v
localhost:8000
  |
  v
[app service: FastAPI + raw HTML]
  |
  | MySQL connection to host "db", port 3306
  v
[db service: MySQL 8.4]
  |
```

```
    v
[mysql-data named volume: /var/lib/mysql]
```

前端仍是 FastAPI 提供的纯 HTML、CSS 和 JavaScript。只有数据库存储
实现发生了变化。

### Compose 主要概念

| YAML 键 | 在我们示例中的含义 |
|---|---|
| `services` | 我们需要的容器：app 和 db |
| `build` | 从我们本地的 Dockerfile 构建应用 |
| `build.args` | 将 BASE_IMAGE 传给 Dockerfile |
| `image` | 拉取已有镜像，如 mysql:8.4 |
| `ports` | 在你电脑的 127.0.0.1:8000 上暴露 FastAPI |
| `environment` | 为容器提供数据库设置 |
| `depends_on` | 在启动应用前等待数据库的健康检查 |
| `healthcheck` | 验证 MySQL 能接受真实查询 |
| `volumes` | 在容器替换时保留 MySQL 文件 |

Compose 会自动创建一个私有项目网络。应用通过服务名 db（而非 localhost）访问 MySQL。
在容器内部，localhost 指的是该容器本身。

构建与拉取：docker compose build app 使用其 Dockerfile 构建 FastAPI 镜像；而 db
服务通常拉取现成的 mysql:8.4 镜像。你无需编写 MySQL 的 Dockerfile。

## 6. 为 MySQL 准备原有的 Python 应用

我们的起始应用使用 app/database.py，它调用 Python 内置的 sqlite3 模块。仅修改
compose.yaml 是不够的：SQLite 代码无法与 MySQL 通信。我们将为现有测试保留 SQLite 后端，
并为 Compose 添加一个小型 MySQL 后端。

以下更改与之前的单元、集成和 API 测试兼容。那些现有测试
将继续使用临时 SQLite 数据库；它们不会自动测试新的 MySQL
后端。

### 步骤 A - 添加 Python MySQL 驱动

打开 requirements.txt，在现有依赖之后添加这一行：

```
PyMySQL[rsa]>=1.1,<2
```

PyMySQL 向 MySQL 发送参数化查询；rsa 附加包支持现代 MySQL 服务器使用的认证方式。
requirements-dev.txt 已经包含 requirements.txt，因此那里
无需改动。

### Step B - Create app/mysql_database.py

复制这个完整文件。它提供与 SQLite 模块相同的六个 CRUD 函数。
_database_path 参数被 MySQL 有意忽略；它使现有的 FastAPI 路由调用
在两种模式下保持一致。

```
"""MySQL CRUD adapter matching the existing SQLite module's public
functions.

The Path argument is retained so the FastAPI routes and SQLite test
fixtures
can use either backend without changing their function calls.
"""

import os
from pathlib import Path

from app.schemas import Car, CarInput


def connect():
    # Import only when the MySQL backend is selected. SQLite-only tests
stay isolated.
    import pymysql
    from pymysql.constants import CLIENT
    from pymysql.cursors import DictCursor

    return pymysql.connect(
        host=os.environ.get("MYSQL_HOST", "db"),
        port=int(os.environ.get("MYSQL_PORT", "3306")),
        user=os.environ["MYSQL_USER"],
        password=os.environ["MYSQL_PASSWORD"],
        database=os.environ["MYSQL_DATABASE"],
        charset="utf8mb4",
        cursorclass=DictCursor,
        autocommit=True,
        client_flag=CLIENT.FOUND_ROWS,
        connect_timeout=10,
    )


def initialize(_database_path: Path) -> None:
    with connect() as connection, connection.cursor() as cursor:
        cursor.execute(
            """
            CREATE TABLE IF NOT EXISTS cars (
                id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
                make VARCHAR(60) NOT NULL,
```

```
                model VARCHAR(60) NOT NULL,
                year INT NOT NULL,
                color VARCHAR(40) NOT NULL DEFAULT '',
                CONSTRAINT valid_make CHECK (CHAR_LENGTH(TRIM(make)) > 0),
                CONSTRAINT valid_model CHECK (CHAR_LENGTH(TRIM(model)) >
0),
                CONSTRAINT valid_year CHECK (year BETWEEN 1886 AND 2100)
            ) CHARACTER SET utf8mb4
            """
        )


def list_cars(_database_path: Path) -> list[Car]:
    with connect() as connection, connection.cursor() as cursor:
        cursor.execute("SELECT id, make, model, year, color FROM cars ORDER
BY id DESC")
        return [Car(**row) for row in cursor.fetchall()]


def get_car(_database_path: Path, car_id: int) -> Car | None:
    with connect() as connection, connection.cursor() as cursor:
        cursor.execute(
            "SELECT id, make, model, year, color FROM cars WHERE id = %s",
(car_id,)
        )
        row = cursor.fetchone()
        return Car(**row) if row else None


def create_car(_database_path: Path, car: CarInput) -> Car:
    with connect() as connection, connection.cursor() as cursor:
        cursor.execute(
            "INSERT INTO cars (make, model, year, color) VALUES (%s, %s,
%s, %s)",
            (car.make, car.model, car.year, car.color),
        )
        return Car(id=cursor.lastrowid, **car.model_dump())


def update_car(_database_path: Path, car_id: int, car: CarInput) -> Car |
None:
    with connect() as connection, connection.cursor() as cursor:
        matched = cursor.execute(
            """UPDATE cars
               SET make = %s, model = %s, year = %s, color = %s
               WHERE id = %s""",
            (car.make, car.model, car.year, car.color, car_id),
        )
        return Car(id=car_id, **car.model_dump()) if matched else None


def delete_car(_database_path: Path, car_id: int) -> bool:
    with connect() as connection, connection.cursor() as cursor:
        deleted = cursor.execute("DELETE FROM cars WHERE id = %s",
```

```
  (car_id,))
        return deleted == 1
```

重要的 SQL 差异：SQLite 使用 ? 作为查询参数占位符，而 PyMySQL 使用 %s。两者都是绑定参数，而非字符串格式化。MySQL 使用 AUTO_INCREMENT，而 SQLite 使用 AUTOINCREMENT。该适配器为每个操作打开并关闭连接；这对训练练习已足够，不是高并发生产模式。

### 步骤 C – 在 app/main.py 中选择后端

1. 将导入从：

```
from app import database
```

改为：

```
from app import database, mysql_database
```

2. 在 create_app() 内、紧跟给 db_path 赋值并以 ).resolve(), add:

```
    # 传入 SQLite 文件路径的测试始终使用 SQLite。
    storage = (
        mysql_database
        if database_path is None and os.getenv("CARS_DB_BACKEND") ==
"mysql"
        else database
    )
```

3. 在 create_app() 中替换以下函数调用（仅这些调用；不要重命名导入本身）：

| 之前 | 之后 |
| --- | --- |
| database.initialize(db_path) | storage.initialize(db_path) |
| database.list_cars(db_path) | storage.list_cars(db_path) |
| database.get_car(db_path, car_id) | storage.get_car(db_path, car_id) |
| database.create_car(db_path, car) | storage.create_car(db_path, car) |
| database.update_car(db_path, car_id, car) | storage.update_car(db_path, car_id, car) |
| database.delete_car(db_path, car_id) | storage.delete_car(db_path, car_id) |

无需修改任何 HTML、JavaScript、schema 或 API 路径。没有 CARS_DB_BACKEND=mysql 时，
应用仍使用 SQLite。现有测试中显式的 create_app(database_path) 调用始终使用
SQLite，与环境变量无关。

### 步骤 D - 保留 Dockerfile

使用之前指南中相同的 Dockerfile（ARG BASE_IMAGE=python:3.12-slim、COPY
requirements.txt、COPY app/、Uvicorn 命令等）。更新后的 requirements.txt 会在 Compose 重建镜像时
安装 MySQL 驱动。

MySQL 无需额外的 Dockerfile；Compose 为数据库服务使用官方 MySQL 镜像。
现有的 SQLite ENV CARS_DB_PATH=/data/cars.db 在 MySQL 模式下无害，
因为 MySQL 适配器不使用该路径。

## 7. 创建 Compose 文件

Create **compose.yaml** at the root of cars-crud/:

```
services:
  app:
    build:
      context: .
      args:
        BASE_IMAGE: ${BASE_IMAGE:-python:3.12-slim}
    ports:
      - "127.0.0.1:8000:8000"
    environment:
      CARS_DB_BACKEND: mysql
      MYSQL_HOST: db
      MYSQL_PORT: "3306"
      MYSQL_DATABASE: ${MYSQL_DATABASE:?Set MYSQL_DATABASE in .env}
      MYSQL_USER: ${MYSQL_USER:?Set MYSQL_USER in .env}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD:?Set MYSQL_PASSWORD in .env}
    depends_on:
      db:
        condition: service_healthy

  db:
    image: ${MYSQL_IMAGE:-mysql:8.4}
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:?Set MYSQL_ROOT_PASSWORD
in .env}
      MYSQL_DATABASE: ${MYSQL_DATABASE:?Set MYSQL_DATABASE in .env}
      MYSQL_USER: ${MYSQL_USER:?Set MYSQL_USER in .env}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD:?Set MYSQL_PASSWORD in .env}
    volumes:
      - mysql-data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysql -h 127.0.0.1 -u\"$$MYSQL_USER\" -
p\"$$MYSQL_PASSWORD\" \"$$MYSQL_DATABASE\" -e 'SELECT 1' >/dev/null 2>&1"]
      interval: 5s
      timeout: 5s
```

```
      retries: 20
      start_period: 30s

volumes:
  mysql-data:
```

### 此文件如何工作

- app.build.context: . 从当前目录的 Dockerfile 构建我们的代码。
- app.ports 让你的电脑可在 http://localhost:8000 访问 FastAPI。127.0.0.1 绑定使这个无认证的训练应用保持本地访问。
- MYSQL_HOST: db 解析到 Compose 网络上的 MySQL 服务。
- db.image 选择官方 MySQL 镜像，而非构建它。
- db.volumes 将 MySQL 数据存储在其常规数据目录 /var/lib/mysql。
- **depends_on.condition: service_healthy** waits for a MySQL health check. A database 容器可能在数据库真正就绪之前就已经运行。
- volumes.mysql-data 声明一个由 Compose 管理的命名卷。
- 健康检查中的双美元符号 $$MYSQL_USER 等，告诉 Compose 保留这些变量，让数据库容器在运行时展开。
- 我们有意不把 MySQL 的 3306 端口发布到宿主机。应用通过内部 Compose 网络连接。

## 8. 创建本地环境文件

在 compose.yaml 旁边创建一个名为 .env 的文件：

```
MYSQL_DATABASE=cars
MYSQL_USER=cars_user
MYSQL_PASSWORD=change_this_training_password
MYSQL_ROOT_PASSWORD=change_this_root_password
```

如果你使用配套参考项目，可以直接将 .env.example 复制为 .env，然后编辑各值。本地使用时请将示例密码替换为你自己的。原项目的 .gitignore 已排除 .env；请确认仍然如此。DOCKER_GUIDE.md 中的 .dockerignore DOCKER_GUIDE.md also excludes it.

Compose 中的 ${NAME:?...} 引用需要来自 .env 或 shell 的变量。.env 是方便的本地配置文件，而非生产密钥库。不要提交它，也不要将其内容发到截图中。docker compose config 会打印解析后的凭据；使用 docker compose config --quiet 可在不显示它们的情况下验证。

### 可选：在中国更快地拉取镜像

如果 Docker Hub 较慢，在所需镜像和标签可通过它获得的情况下，可以使用课程的 Azure 注册表代理。添加到 .env：

```
BASE_IMAGE=efreiwut.azurecr.io/cache/library/python:3.12-slim
# 仅当代理也提供此 MySQL 镜像时：
MYSQL_IMAGE=efreiwut.azurecr.io/cache/library/mysql:8.4
```

不覆盖时的默认值：Dockerfile 使用 python:3.12-slim，Compose 使用 mysql:8.4。
课程代理是拉取镜像的替代来源，而不是保存我们的
数据库或应用镜像的目的地。如果所需镜像无法通过代理获得，可以移除任一覆盖。

## 9. 构建并启动两个服务

如果旧的独立 SQLite 容器仍占用 8000 端口，请确保先停止它：

```
docker stop cars-app
```

如果 cars-app 未运行，可跳过此步。

检查 Compose 安装和 YAML 配置：

```
docker compose version
docker compose config --quiet
```

构建 FastAPI 镜像：

```
docker compose build app
```

启动两个服务（--build 选项在需要时也会构建应用镜像）：

```
docker compose up --build -d
```

Docker Compose 将：

1. 使用 Dockerfile 和更新后的 Python 依赖构建应用镜像。
2. 如果本地没有 db 镜像则拉取它。
3. 创建共享网络和 mysql-data 卷。
4. 启动 MySQL 并等待其健康检查。
5. 启动 FastAPI，必要时创建 cars 表。

访问 http://localhost:8000/。创建、查看、编辑和删除车辆。使用 http://localhost:8000/docs 直接测试 REST API。

检查运行中的服务和日志：

```
docker compose ps
docker compose logs -f app
docker compose logs -f db
```

按 Ctrl+C 停止查看日志。这不会停止容器。

要直接在 MySQL 中查看记录（提示时输入 MYSQL_PASSWORD）：

```
docker compose exec db mysql -u cars_user -p cars
```

然后在 MySQL 提示符中运行 SELECT * FROM cars; 并退出。如果你在 .env 中修改了 MYSQL_USER 或 MYSQL_DATABASE 值，请改用那些值。

## 10. 证明 MySQL 的持久性

通过网站保存至少一辆车后：

```
docker compose down
docker compose up -d
docker compose ps
```

刷新 http://localhost:8000/。你的车辆应当仍在，因为命名 MySQL 卷在 `docker compose down`.

危险：下面的操作是故意破坏性的，不属于持久性检查的一部分：

```
docker compose down -v
```

-v 会连同容器一起删除已声明的命名卷。对本教程而言，这意味着删除 MySQL 数据库内容。只有当你明确想重置所有练习数据时才使用。在 .env 中修改 MySQL 初始化密码或数据库名，不会重新初始化已填充数据的 MySQL 数据卷。

## 11. 关键 Compose 命令

| 命令 | 用途 |
| --- | --- |
| `docker compose config --quiet` | 验证 Compose YAML 和所需变量 |
| `docker compose build app` | 仅构建/重建 FastAPI 镜像 |
| `docker compose build --no-cache app` | 不使用缓存层重建 FastAPI |
| `docker compose pull db` | 拉取所选 MySQL 镜像 |

| 命令 | 用途 |
| --- | --- |
| `docker compose up -d` | 在后台启动服务 |
| `docker compose up --build -d` | 构建所需镜像，然后启动服务 |
| `docker compose ps` | 显示服务/容器状态 |
| `docker compose images` | 显示服务使用的镜像 |
| `docker compose logs -f app` | 跟踪 FastAPI 日志 |
| `docker compose logs -f db` | 跟踪 MySQL 日志 |
| `docker compose exec app sh` | 在 FastAPI 容器中打开 shell |
| `docker compose exec db mysql -u cars_user -p cars` | 在其容器内连接 MySQL |
| `docker compose stop` | 停止服务但不删除容器 |
| `docker compose down` | 删除 Compose 容器和网络，保留命名卷 |
| `docker compose down -v` | 删除容器和命名卷（数据丢失） |

`build:` 与 `image:`：`build` 从你的代码和 `Dockerfile` 创建镜像。`image` 按名称引用镜像，通常用于拉取。如果你希望 Compose 给构建出的应用一个特定标签，可以在 `app` 服务下添加 `image: cars-crud:mysql`。该名称是本地名称，除非你自己打标签并推送到注册表。

## 12. 故障排查

| 问题 | 可能的原因 / 修复 |
| --- | --- |
| 8000 端口已被占用 | 停止旧的 `cars-app` 或本地 Uvicorn 服务器；或将宿主机 port to `8001:8000`. |
| `ModuleNotFoundError: pymysql` | Add `PyMySQL[rsa]>=1.1,<2` to `requirements.txt` and run `docker compose up --build -d`. |
| 应用无法连接 MySQL | Check `docker compose ps`, `docker compose logs db`, 凭据，以及 `MYSQL_HOST: db`。 |
| MySQL 一直不健康 | 查看 `docker compose logs db` 及其初始化/错误；给首次启动留出时间。 |
| 新的 `.env` 密码不生效 | 已有的 MySQL 卷保留原来的用户/密码；环境变量只初始化新的数据目录。 |
| SQLite 中的车辆不出现在 MySQL 中 | 它们是两个独立的数据库。导出/导入是单独的迁移任务。 |

| 问题 | 可能的原因 / 修复 |
| --- | --- |
| `Permission denied` on SQLite 文件 | 在 /data 挂载命名卷，并使用现有 Dockerfile 的 /data 所有权设置。 |
| 无法拉取 mysql:8.4 | 确认 Docker Hub 可访问；仅当该镜像/标签受支持时才使用 Azure 代理。 |
| MySQL 可从外部访问 | 除非本地工作需要，否则不要为 db 添加宿主机端口映射。 |

## 13. 快速回顾

- 卷让状态在容器替换后得以保留。SQLite 使用 cars-data:/data；MySQL 使用 `mysql-data:/var/lib/mysql`.
- Dockerfile 定义如何构建一个镜像。
- Compose 文件描述如何从构建好的或已有的镜像运行多个协同服务。

- 我们的 HTML/前端与 FastAPI API 同在一个 app 服务中；MySQL 运行在单独的 db 服务中。
- `docker compose up --build -d` builds/starts the system; `docker compose down` 停止/删除它而不删除命名卷。

官方参考

- Docker volumes
- Docker Compose Quickstart
- Compose build specification
- Startup order and health checks
- Compose variables and .env
- MySQL Docker Official Image